SVM and Logistic Regression for Wine-Quality Prediction

Ettore Oteri, 34054A DSE

31 August 2025

1 Introduction

The purpose of this project is the implementation and comparative analysis of two classic binary classification algorithms, Logistic Regression (LR) and Support Vector Machine (SVM), both developed entirely from scratch. The task at hand is wine quality prediction, defined as a binary variable: wines are considered "good" if their quality score is greater than or equal to 6, and "bad" if their quality score is below 6. This project lies within the framework of supervised machine learning, with a strong emphasis not only on the correct implementation of the models, but also on following a rigorous methodology for statistical analysis, data preprocessing, and experimental validation. Furthermore, in order to explore the models' ability to capture non-linear relationships, both algorithms have been extended to kernelized versions (KLR and KSVM), thus allowing a comparison between linear and non-linear approaches in terms of predictive performance.

2 About the Dataset

The dataset used is the Wine Quality Dataset (UCI Machine Learning Repository), which contains both red and white wine samples. Each observation is described by 11 physicochemical variables (such as fixed acidity, residual sugar, pH, and sulphates) and by a quality score assigned by experts on a scale from 0 to 10. In total, the dataset includes 6,497 samples (1,599 red wines and 4,898 white wines). For the purpose of this project, the target variable was binarized by labeling wines as "good" (quality ≥ 6) or "bad" (quality ¡ 6), resulting in a moderately imbalanced but manageable distribution. During preprocessing, red and white wines were merged into a single dataset, features were normalized to ensure comparability, and the data was split into training and test sets in a stratified manner, while ensuring that no test set information was used during modeling or hyperparameter tuning.

3 Logistic Regression

Logistic Regression is a linear binary classification model that estimates the probability of class membership through the logistic (sigmoid) function. The objective is to optimize the linear coefficients by minimizing the cross-entropy loss, optionally including an L2 regularization term to reduce overfitting. This formulation allows the model to adapt to binary data and produce interpretable predictions as probabilities. In practice, the loss minimization is performed using gradient descent, a concept that is later implemented directly in the code.

4 Support Vector Machine

The linear SVM is a classification model that seeks the optimal hyperplane separating classes with the maximum possible margin. The objective function combines the hinge loss, which penalizes misclassified observations, with an L2 regularization term on the weights to control model complexity. The points that define the optimal hyperplane are called support vectors. Optimization can be performed using gradient descent, and the linear SVM serves as a baseline to evaluate the effectiveness of kernel methods.

5 Kernel Methods

Kernel methods extend linear models, such as LR and SVM, to non-linear problems by implicitly mapping features into a higher-dimensional space via a kernel function, such as the Radial Basis Function (RBF). Kernel Logistic Regression maintains the cross-entropy loss, while Kernel SVM maintains the hinge loss, both in a kernelized form. The kernel parameter gamma controls the model's flexibility: small values produce smoother, more global decision boundaries, while large values produce more local and complex boundaries. These theoretical concepts are then reflected in the code through the management of kernel parameters and the construction of kernel matrices for non-linear predictions.

6 Code

I start with importing a set of fundamental libraries for machine learning in Python. Specifically, NumPy and Pandas were used for data handling, respectively for numerical operations on matrices and vectors and for the manipulation of tabular datasets. For graphical analysis, Matplotlib and Seaborn were employed, enabling the production of effective visualizations, for instance when analyzing performance metrics or confusion matrices. In addition, the Scikit-learn package was used just to properly split the data (train/test split, K-Fold cross-validation) and to compute the main classification evaluation metrics such as accuracy, precision, recall, and F1-score. Finally, SciPy offered additional

mathematical tools, in particular the distance computation function, which is essential for the implementation of kernel methods.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split,
    KFold
from sklearn.metrics import accuracy_score,
    precision_score, recall_score, f1_score,
    confusion_matrix, classification_report
from scipy.spatial.distance import cdist
```

The dataset was loaded from the CSV files corresponding to red and white wines, available in the UCI Machine Learning Repository. After import, a categorical variable named type was added, with value 0 for red wines and 1 for white wines, in order to keep track of the wine type within a single unified dataset. Finally, the two datasets were concatenated along the rows and the index was reset. This step provides a single dataset containing all observations, thus simplifying the subsequent preprocessing, training, and evaluation phases of the models.

```
red = pd.read_csv("/Users/ettoreoteri/Desktop/wine+
    quality/winequality-red.csv", sep=";")
white = pd.read_csv("/Users/ettoreoteri/Desktop/wine+
    quality/winequality-white.csv", sep=";")
red["type"] = 0; white["type"] = 1
data = pd.concat([red, white], axis=0).reset_index(
    drop=True)
```

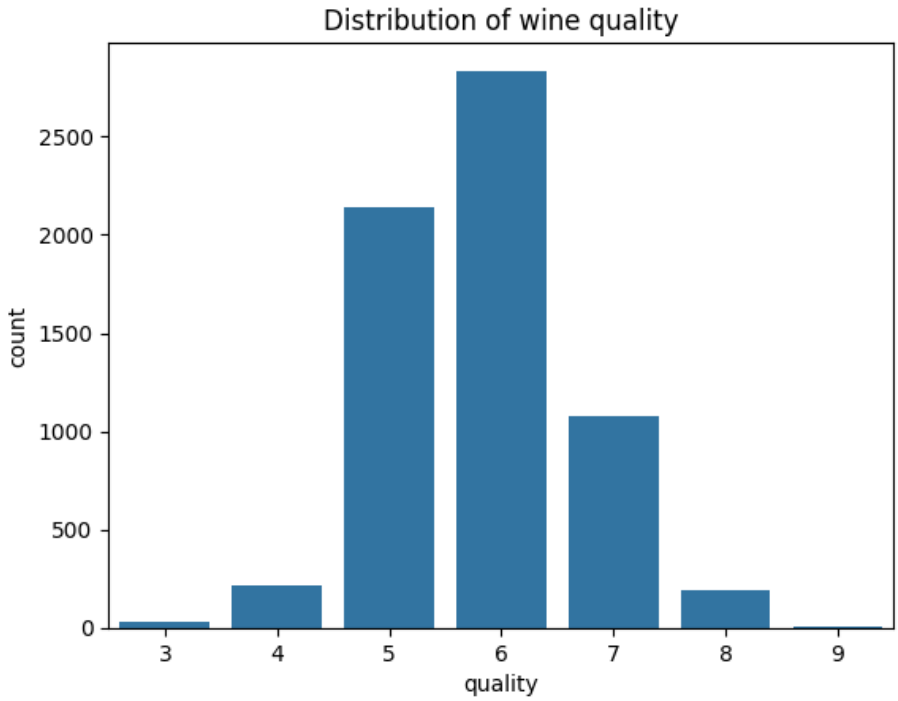
This block of code performs some exploratory analysis. First, it prints the dataset's descriptive statistics with data.describe(). Then, a bar plot is generated using sns.countplot to show the distribution of the quality variable. To transform the task into a binary classification problem, a new column called label is added, which takes value 1 if wine quality is greater or equal to 6, and 0 otherwise. The distribution of this new variable is also displayed with a bar plot. Two additional plots are then created: a histogram of the alcohol variable, separated by label, and a scatterplot showing the relationship between volatile acidity and citric acid, again colored according to the label variable. These visualizations are used to graphically represent the data and their relationships.

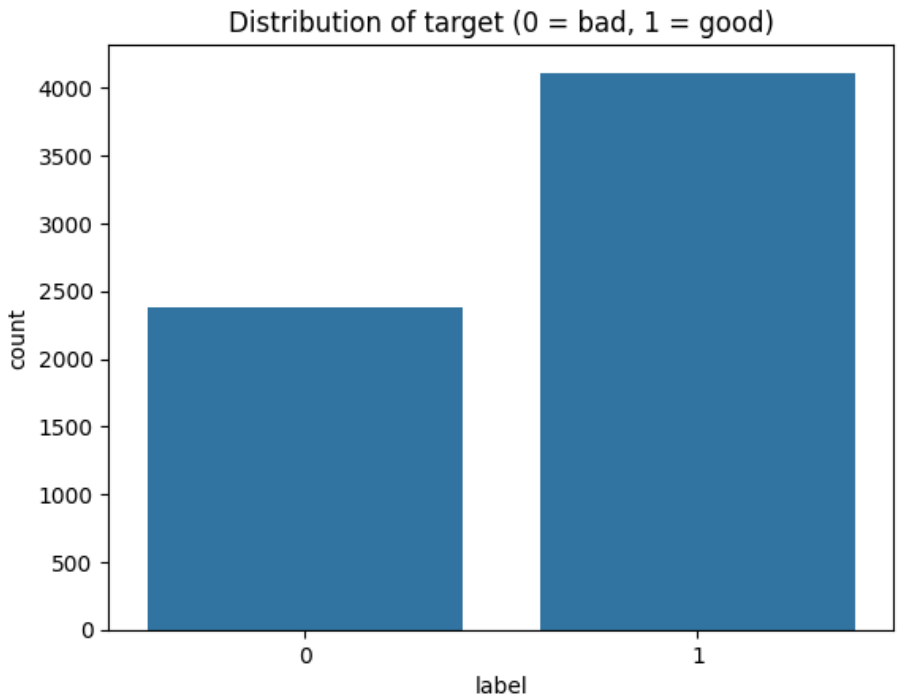
```
print("=== Descriptive statistics ===")
print(data.describe())

sns.countplot(x="quality", data=data)
plt.title("Distribution of wine quality")
plt.show()

data["label"] = (data["quality"] >= 6).astype(int)
sns.countplot(x="label", data=data)
plt.title("Distribution of target (0 = bad, 1 = good)"
    )
plt.show()

```

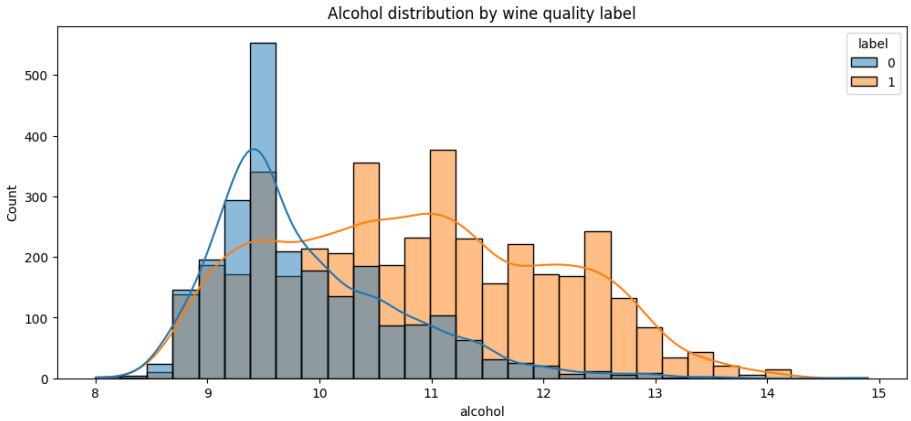
```python
plt.figure(figsize=(12,5))
sns.histplot(data=data, x="alcohol", hue="label", kde=True, bins=30)
plt.title("Alcohol distribution by wine quality label")
plt.show()

sns.scatterplot(x="volatile acidity", y="citric acid", hue="label", data=data, alpha=0.5)
plt.title("Scatterplot: Volatile acidity vs Citric acid by label")
plt.show()
```

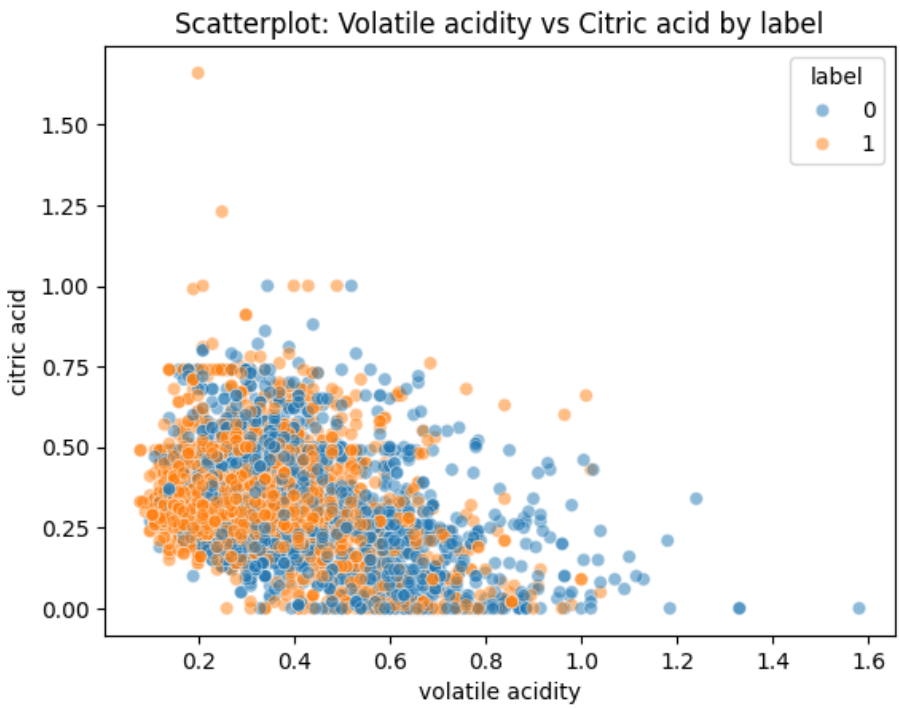
Output:

```
=== Descriptive statistics ===
       fixed acidity  volatile acidity  citric acid  residual sugar  \
count    6497.000000       6497.000000  6497.000000     6497.000000
mean        7.215307          0.339666     0.318633        5.443235
std         1.296434          0.164636     0.145318        4.757804
min         3.800000          0.080000     0.000000        0.600000
25%         6.400000          0.230000     0.250000        1.800000
50%         7.000000          0.290000     0.310000        3.000000
75%         7.700000          0.400000     0.390000        8.100000
max        15.900000          1.580000     1.660000       65.800000

         chlorides  free sulfur dioxide  total sulfur dioxide      density  \
count  6497.000000          6497.000000           6497.000000  6497.000000
mean      0.056034            30.525319            115.744574     0.994697
std       0.035034            17.749400             56.521855     0.002999
min       0.009000             1.000000              6.000000     0.987110
25%       0.038000            17.000000             77.000000     0.992340
50%       0.047000            29.000000            118.000000     0.994890
75%       0.065000            41.000000            156.000000     0.996990
max       0.611000           289.000000            440.000000     1.038980

                pH    sulphates      alcohol      quality         type
count  6497.000000  6497.000000  6497.000000  6497.000000  6497.000000
mean      3.218501     0.531268    10.491801     5.818378     0.753886
std       0.160787     0.148806     1.192712     0.873255     0.430779
min       2.720000     0.220000     8.000000     3.000000     0.000000
25%       3.110000     0.430000     9.500000     5.000000     1.000000
50%       3.210000     0.510000    10.300000     6.000000     1.000000
75%       3.320000     0.600000    11.300000     6.000000     1.000000
max       4.010000     2.000000    14.900000     9.000000     1.000000
```


Distribution of wine quality
count
quality
0
500
1000
1500
2000
2500
3
4
5
6
7
8
9


Distribution of target (0 = bad, 1 = good)
count
label
0
500
1000
1500
2000
2500
3000
3500
4000
0
1





In this block the dataset is prepared for model training. First, the features are separated from the outputs: all columns except quality and label are kept as independent variables, while the label column becomes the target. The data is then split into training and test sets, with an 80/20 ratio and stratification to preserve the target distribution in both subsets. Finally, feature normalization is applied: the mean and standard deviation are computed on the training set, and both subsets are standardized accordingly. This ensures that the models

are trained on normalized data and that the test set is transformed using the same parameters from the training set, preventing bias.

```
X = data.drop(["quality", "label"], axis=1).values
y = data["label"].values

X_train, X_test, y_train, y_test = train_test_split(X,
     y, test_size=0.2,
random_state=42, stratify=y)
mean, std = X_train.mean(axis=0), X_train.std(axis=0)
X_train = (X_train - mean) / std
X_test = (X_test - mean) / std
```

In this block a custom Logistic Regression class is implemented from scratch. During initialization, the main hyperparameters are defined: the learning rate, the number of training iterations, and the L2 regularization coefficient. A stable sigmoid function is also included, applying the sigmoid function in a numerically stable way by clipping extreme values to avoid overflow.

The fit method manages the training process: weights are initialized to zero and updated iteratively using gradient descent. At each step, predictions are computed, gradients with respect to weights and bias are calculated, and parameters are updated with regularization to mitigate overfitting. Additionally, the logistic loss and the training accuracy are recorded at every iteration, allowing monitoring of the learning progress.

The predict method applies the sigmoid function to compute probabilities and classifies a sample as positive if the estimated probability is at least 0.5. This provides a complete implementation of Logistic Regression, including training with convergence tracking and prediction capabilities.

```
class LogisticRegression:
    def __init__(self, lr=0.01, n_iters=300,
        reg_lambda=0.01):
        self.lr, self.n_iters, self.reg_lambda = lr,
            n_iters, reg_lambda

    def stable_sigmoid(self, z):
        z = np.clip(z, -500, 500)
        return 1 / (1 + np.exp(-z))

    def fit(self, X, y):
        n, d = X.shape
        self.w = np.zeros(d)
        self.b = 0
        self.losses = []
        self.train_acc = []

        for _ in range(self.n_iters):
            linear = X @ self.w + self.b
            pred = self.stable_sigmoid(linear)
            dw = X.T @ (pred - y) / n + self.
                reg_lambda * self.w
            db = (pred - y).mean()
            self.w -= self.lr * dw
            self.b -= self.lr * db
```

```
            loss = -np.mean(y * np.log(pred + 1e-9) + (1 - y) * np.log(1 - pred + 1e-9))
            self.losses.append(loss)
            self.train_acc.append(accuracy_score(y, (pred >= 0.5).astype(int)))

    def predict(self, X):
        return (self.stable_sigmoid(X @ self.w + self.b) >= 0.5).astype(int)
```

This block implements a linear Support Vector Machine from scratch. The SVM class defines the main model parameters: learning rate (lr), regularization coefficient (lam), and number of iterations (n-iters).

The fit method trains the model using stochastic gradient descent to minimize the hinge loss typical of SVMs. For each training sample, the margin condition is checked: if satisfied, only regularization is applied; otherwise, the weight vector and bias are updated according to the margin violation. At each iteration, the mean loss and training accuracy are recorded to monitor training progress.

The predict method returns binary predictions by comparing the dot product between weights and features with the bias. This implementation allows observing the evolution of weights and performance during training without relying on external libraries.

```
class SVM:
    def __init__(self, lr=0.001, lam=0.01, n_iters=300):
        self.lr = lr
        self.lam = lam
        self.n_iters = n_iters
        self.losses = []
        self.train_acc = []

    def fit(self, X, y):
        n,d = X.shape
        y_ = np.where(y==1,1,-1)
        self.w = np.zeros(d)
        self.b = 0

        for _ in range(self.n_iters):
            loss = 0
            for i,x in enumerate(X):
                cond = y_[i]*(x@self.w - self.b) >= 1
                if cond:
                    self.w -= self.lr*(2*self.lam*self.w)
                else:
                    self.w -= self.lr*(2*self.lam*self.w - y_[i]*x)
                    self.b -= self.lr*y_[i]
                loss += max(0,1-y_[i]*(x@self.w-self.b))
            self.losses.append(loss/n)

            y_pred_train = self.predict(X)
```

```
            self.train_acc.append(np.mean(y_pred_train == y))

    def predict(self, X):
        return ((X@self.w - self.b) >= 0).astype(int)
```

This function implements the construction of the RBF (Radial Basis Function) kernel matrix, a technique that allows extending linear models, such as SVM and logistic regression, to non-linear contexts. The function takes two datasets, X1 and X2, as input and computes the squared Euclidean distance between all pairs of points. These distances are then transformed using the negative exponential function, controlled by the parameter gamma.

The result is a symmetric matrix where each element represents the similarity between two observations: values close to 1 indicate highly similar points in feature space, while values close to 0 indicate distant ones. This approach enables the model to operate implicitly in a higher-dimensional space without explicitly computing it, leveraging the so-called "kernel trick."

```
def rbf_kernel_matrix(X1, X2, gamma=0.5):
    dists = cdist(X1, X2, 'sqeuclidean')
    return np.exp(-gamma * dists)
```

This class implements Kernel Logistic Regression (KLR), a non-linear extension of logistic regression that leverages the RBF kernel to handle complex relationships in the data. Unlike standard logistic regression, which works directly with feature weights, this model relies on coefficients alpha associated with the training samples and uses the kernel matrix to represent similarities between observations.

During training, the kernel matrix of the training data is computed, and the alpha values are updated iteratively via gradient descent. Predictions are passed through the stable sigmoid function, which ensures numerical stability for very large or very small inputs. At each iteration, both the logistic loss and the training accuracy are recorded, allowing one to track the progress of the training process.

In the prediction phase, the model computes the kernel matrix between new data and the training set, evaluates probabilities through the linear combination with alpha, and applies a 0.5 threshold to produce final class predictions. This approach allows the model to capture non-linear decision boundaries in the original feature space, making KLR particularly effective in complex classification problems.

```
class KernelLogisticRegression:
    def __init__(self, kernel=rbf_kernel_matrix, lr=0.001, n_iters=150, gamma=0.5):
        self.kernel, self.lr, self.n_iters, self.gamma = kernel, lr, n_iters, gamma

    def stable_sigmoid(self, z):
        z = np.clip(z, -500, 500)
        return 1 / (1 + np.exp(-z))

    def fit(self, X, y):
```

```
        self.X = X
        self.y = y
        n = len(y)
        self.alpha = np.zeros(n)
        self.losses = []
        self.train_acc = []

        K = self.kernel(X, X, gamma=self.gamma)
        for _ in range(self.n_iters):
            z = self.alpha @ K
            pred = self.stable_sigmoid(z)
            grad = pred - y
            self.alpha -= self.lr * grad
            loss = -np.mean(y * np.log(pred + 1e-9) +
                (1 - y) * np.log(1 - pred + 1e-9))
            self.losses.append(loss)
            self.train_acc.append(accuracy_score(y, (
                pred >= 0.5).astype(int)))

    def predict(self, Z):
        K_test = self.kernel(Z, self.X, gamma=self.
            gamma)
        pred = self.stable_sigmoid(self.alpha @ K_test
            .T)
        return (pred >= 0.5).astype(int)
```

This class implements a Kernel Support Vector Machine (KSVM), a non-linear version of SVM that leverages the RBF kernel to represent data in a higher-dimensional feature space. The principle is similar to Kernel Logistic Regression: instead of working directly in the original feature space, the model relies on a kernel matrix that measures similarities between observations.

During training, the model maintains a vector of coefficients alpha associated with the training samples. At each iteration, the margin condition is checked for every sample: if the condition is not satisfied, the corresponding alpha is updated. This process effectively pushes the model to maximize the margin between classes. The hinge loss, typical of SVMs, is computed to quantify the penalty for misclassified instances or those close to the margin, while training accuracy is also tracked to monitor progress.

In the prediction phase, the kernel matrix between new samples and the training set is computed. The final classification is obtained through the sign of the linear combination of the alpha coefficients with the training labels and the kernel similarities. This allows the model to capture non-linear decision boundaries, making KSVM suitable for complex problems where a linear classifier would be insufficient.

```
class KernelSVM:
    def __init__(self,kernel=rbf_kernel_matrix,lr
        =0.001,n_iters=150,gamma=0.5):
        self.kernel = kernel
        self.lr = lr
        self.n_iters = n_iters
        self.gamma = gamma
        self.alpha = None
        self.losses = []
```

```
        self.train_acc = []

    def fit(self,X,y):
        self.X = X
        self.y = np.where(y==1,1,-1)
        n = len(y)
        self.alpha = np.zeros(n)
        K = self.kernel(X,X,gamma=self.gamma)

        for _ in range(self.n_iters):
            for i in range(n):
                cond = self.y[i]*(self.alpha*self.y)@K[:,i]>=1
                if not cond: self.alpha[i]+=self.lr

            margin = self.y * (K @ (self.alpha * self.y))
            hinge_loss = np.mean(np.maximum(0, 1-margin))
            self.losses.append(hinge_loss)

            y_pred_train = self.predict(X)
            self.train_acc.append(np.mean(y_pred_train == y))

    def predict(self,Z):
        K_test = self.kernel(Z,self.X,gamma=self.gamma)
        pred = (self.alpha*self.y) @ K_test.T
        return (pred >= 0).astype(int)
```

This function implements a manual grid search procedure to optimize the hyperparameters of the models. It relies on K-Fold cross-validation (3 splits), which randomly partitions the dataset into training and validation sets in a reproducible way.

The function iterates over all possible combinations of hyperparameters specified in params (such as lr, lam, gamma). For each configuration, it trains the corresponding model (Logistic Regression, SVM, Kernel Logistic Regression, Kernel SVM) on the training folds and evaluates its performance on the validation fold. The chosen metric is the F1-score, which is particularly suitable for imbalanced datasets since it balances precision and recall.

At the end of the process, the combination of hyperparameters that achieved the highest average F1-score across validation folds is selected, printed, and returned. This way, the function automates hyperparameter tuning and improves predictive performance without relying on external machine learning libraries.

```
def grid_search(model_name, X, y, params):
    kf = KFold(n_splits=3, shuffle=True, random_state=42)
    best_f1 = -1
    best_params = None

    for lr in params.get("lr", [0.01]):
        for lam in params.get("lam", [0.01]):
            for gamma in params.get("gamma", [0.5]):
```

```
                f1s = []
                for tr, va in kf.split(X):
                    if model_name=="LR":
                        m = LogisticRegressionScratch(lr=lr, reg_lambda=lam, n_iters=200)
                    elif model_name=="SVM":
                        m = SVM(lr=lr, lam=lam, n_iters=300)
                    elif model_name=="KLR":
                        m = KernelLogisticRegression(lr=lr, n_iters=100, gamma=gamma)
                    elif model_name=="KSVM":
                        m = KernelSVM(lr=lr, n_iters=100, gamma=gamma)

                    m.fit(X[tr], y[tr])
                    preds = m.predict(X[va])
                    f1s.append(f1_score(y[va], preds))

                avg_f1 = np.mean(f1s)
                if avg_f1 > best_f1:
                    best_f1 = avg_f1
                    best_params = {"lr": lr, "lam": lam, "gamma": gamma}

    print(f"{model_name} Best params: {best_params}, F1={best_f1:.3f}")
    return best_params
```

This evaluate function provides a complete evaluation of a classification model. It first trains the model on the training data and computes predictions on the test set. Then, standard performance metrics are calculated, including accuracy, precision, recall, F1-score, and the confusion matrix, which are printed along with a detailed classification report.

The function also produces useful visualizations: a heatmap of the confusion matrix, a plot of the training loss (if the model tracks it), and a comparison between training and test accuracy. Additionally, an error analysis is performed by computing false positives and false negatives, along with Type I and Type II errors.

Finally, an automatic comment on the model fit is provided, highlighting potential overfitting or underfitting based on the difference between training and test accuracy. The function returns the predictions on the test set.

```
def evaluate(model,X_train,y_train,X_test,y_test):
    model.fit(X_train,y_train)
    preds=model.predict(X_test)

    acc_train = accuracy_score(y_train, model.predict(X_train))
    acc_test = accuracy_score(y_test, preds)
    prec=precision_score(y_test,preds)
    rec=recall_score(y_test,preds)
    f1=f1_score(y_test,preds)
```

```python
    cm=confusion_matrix(y_test,preds)

    print(f"{model.__class__.__name__}: train_acc={acc_train:.3f}, test_acc={acc_test:.3f}, prec={prec:.3f}, rec={rec:.3f}, f1={f1:.3f}")
    print("\nClassification report:\n", classification_report(y_test, preds))

    sns.heatmap(cm,annot=True,fmt="d",cmap="Blues")
    plt.title(model.__class__.__name__+" Confusion Matrix")
    plt.show()

    if hasattr(model,"losses") and len(model.losses)>0:
        plt.plot(model.losses)
        plt.title(model.__class__.__name__+" Loss")
        plt.show()

    if hasattr(model,"train_acc"):
        plt.plot(model.train_acc, label="Train Acc")
        plt.axhline(y=acc_test, color="red", linestyle="--", label="Final Test Acc")
        plt.legend(); plt.title(model.__class__.__name__+" Training vs Test Accuracy")
        plt.show()

    false_pos = np.where((preds==1)&(y_test==0))[0]
    false_neg = np.where((preds==0)&(y_test==1))[0]
    print(f"False Positives: {len(false_pos)}, False Negatives: {len(false_neg)}")
    print(f"Type I error (FP rate): {len(false_pos)/len(y_test):.3f}")
    print(f"Type II error (FN rate): {len(false_neg)/len(y_test):.3f}")

    if acc_train - acc_test > 0.1:
        print("Possible Overfitting: train accuracy much higher than test.")
    elif acc_train < 0.65 and acc_test < 0.65:
        print("Possible Underfitting: both train and test accuracy are low.")
    else:
        print("Model seems reasonably fit.")

    return preds
```

In this block, the automatic search for the best hyperparameters is performed for each model using the previously defined grid search function.

```python
best_lr_params = grid_search("LR", X, y, {"lr":[0.01,0.05], "lam":[0.001,0.01]})
best_svm_params = grid_search("SVM", X, y, {"lr":[0.001,0.005], "lam":[0.001,0.01]})
best_klr_params = grid_search("KLR", X, y, {"lr":[0.001,0.01], "gamma":[0.1,0.5]})
best_ksvm_params = grid_search("KSVM", X, y, {"lr":[0.001,0.005], "gamma":[0.1,0.5]})
```

In this block, all models are evaluated using the best hyperparameter combinations identified through grid search. For each model — Logistic Regression, SVM, Kernel Logistic Regression, and Kernel SVM — an instance is created with the optimal parameters, and the evaluate function is called.

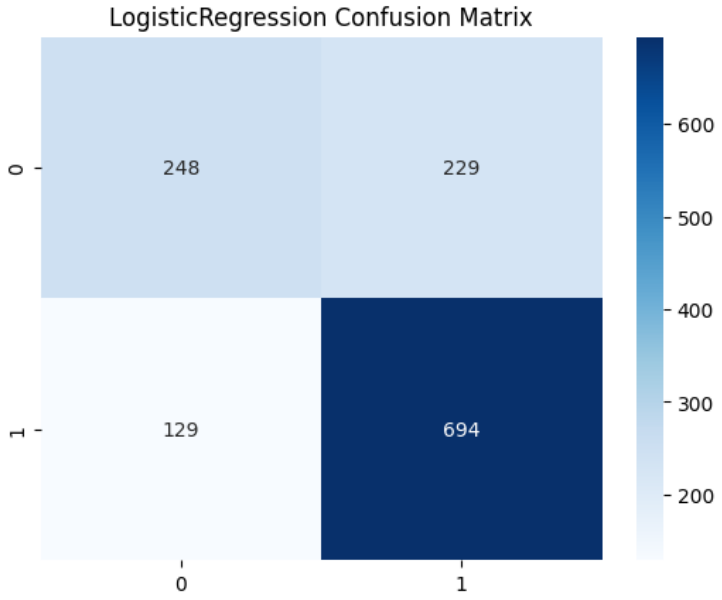
The function trains the model on the training set, computes predictions on the test set, and generates standard performance metrics (accuracy, precision, recall, F1-score) along with the confusion matrix. It also produces visualizations of the training loss and a comparison between training and test accuracy, and performs an error analysis (false positives and false negatives). Finally, it provides a comment on model fit, highlighting potential overfitting or underfitting.

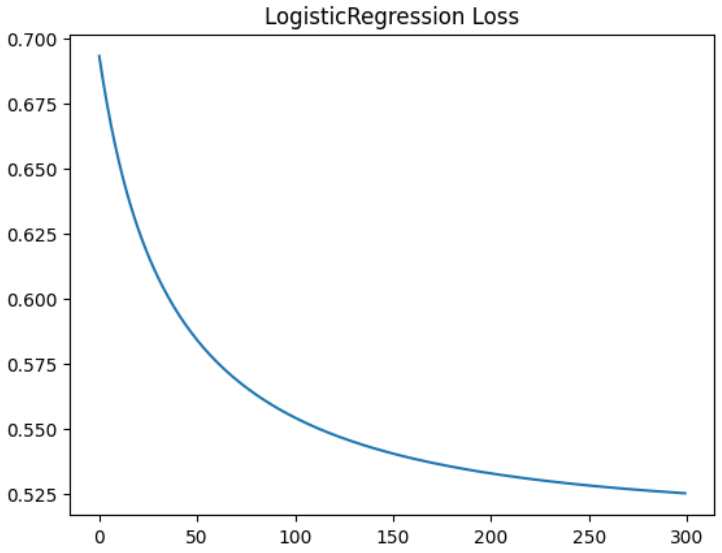
This process provides a comprehensive and comparative evaluation of the four models, allowing identification of the best-performing model on the Wine Quality dataset.

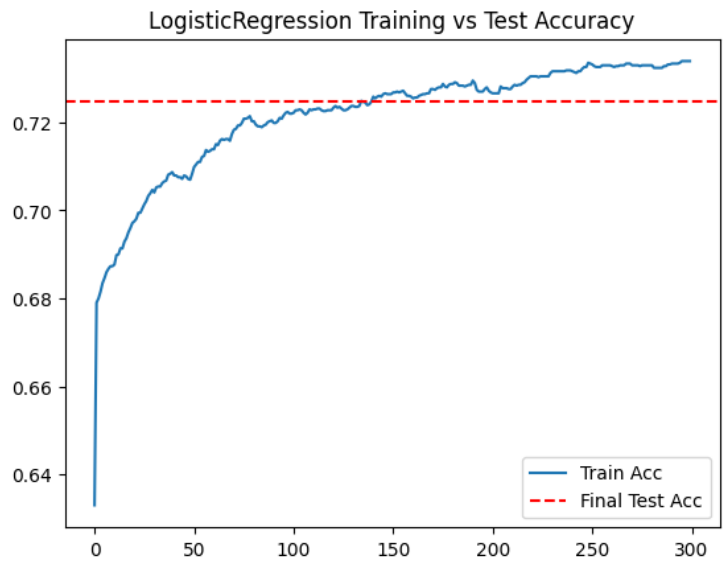
```
evaluate(LogisticRegression(lr=best_lr_params['lr'],
    reg_lambda=best_lr_params['lam']),
         X_train,y_train,X_test,y_test) ;
evaluate(SVM(lr=best_svm_params['lr'], lam=
    best_svm_params['lam']),
         X_train,y_train,X_test,y_test) ;
evaluate(KernelLogisticRegression(lr=best_klr_params['
    lr'], gamma=best_klr_params['gamma']),
         X_train,y_train,X_test,y_test) ;
evaluate(KernelSVM(lr=best_ksvm_params['lr'], gamma=
    best_ksvm_params['gamma']),
         X_train,y_train,X_test,y_test) ;
```

Output:

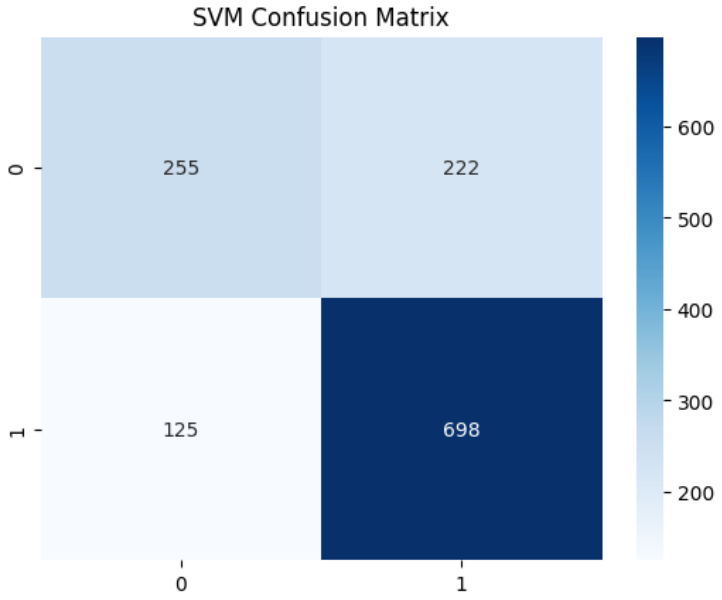
```
LogisticRegression: train_acc=0.734, test_acc=0.725, prec=0.752, rec=0.843, f1=0.795

Classification report:
              precision    recall  f1-score   support

           0       0.66      0.52      0.58       477
           1       0.75      0.84      0.79       823

    accuracy                           0.72      1300
   macro avg       0.70      0.68      0.69      1300
weighted avg       0.72      0.72      0.72      1300
```

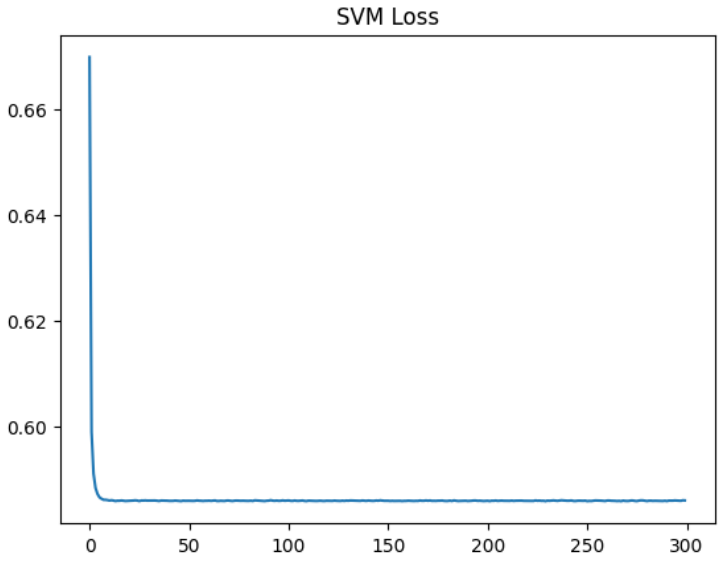

LogisticRegression Confusion Matrix
248
229
129
694
0
1
0
1
600
500
400
300
200

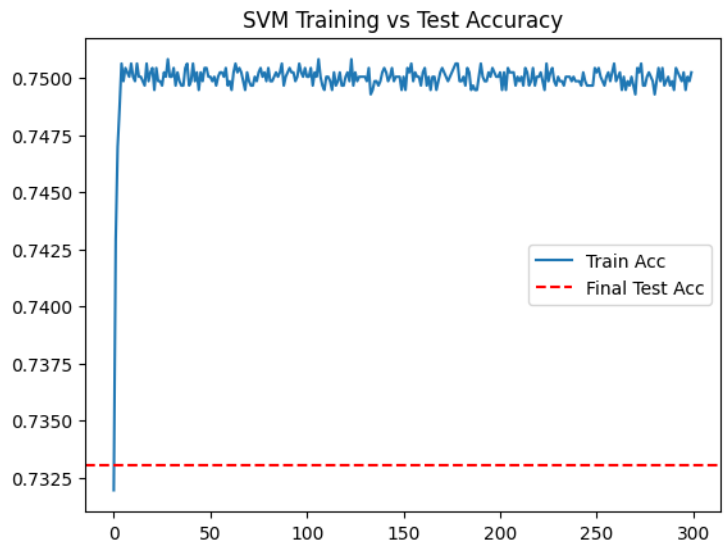

LogisticRegression Loss
0.700
0.675
0.650
0.625
0.600
0.575
0.550
0.525
0
50
100
150
200
250
300



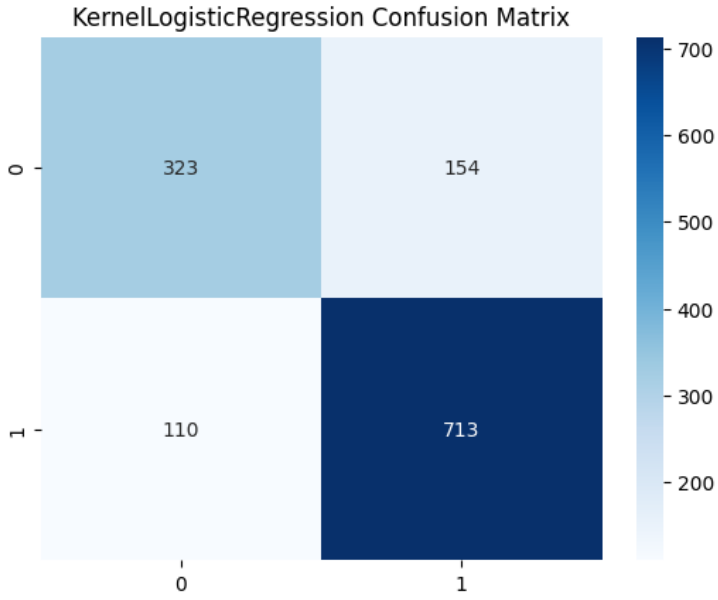

```
False Positives: 229, False Negatives: 129
Type I error (FP rate): 0.176
Type II error (FN rate): 0.099
Model seems reasonably fit.

------------------------------------

SVM: train_acc=0.750, test_acc=0.733, prec=0.759, rec=0.848, f1=0.801

Classification report:
              precision    recall  f1-score   support

           0       0.67      0.53      0.60       477
           1       0.76      0.85      0.80       823

    accuracy                           0.73      1300
   macro avg       0.71      0.69      0.70      1300
weighted avg       0.73      0.73      0.73      1300
```

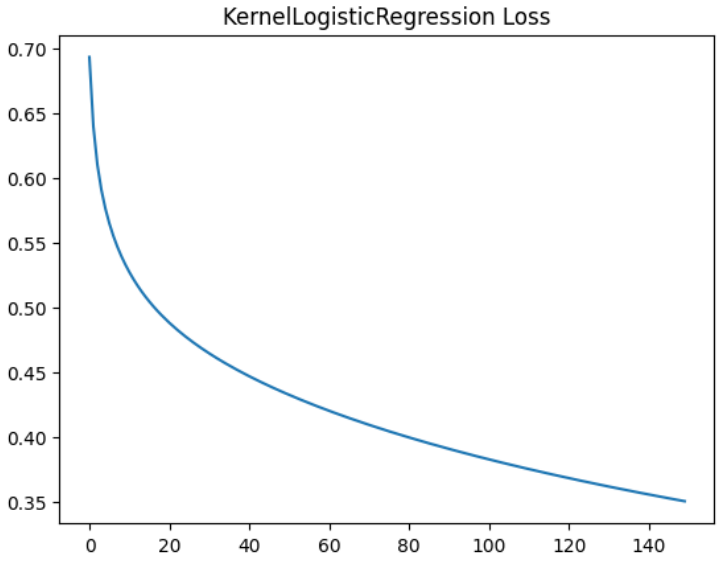

SVM Confusion Matrix
255
222
125
698
0
1
0
1
600
500
400
300
200

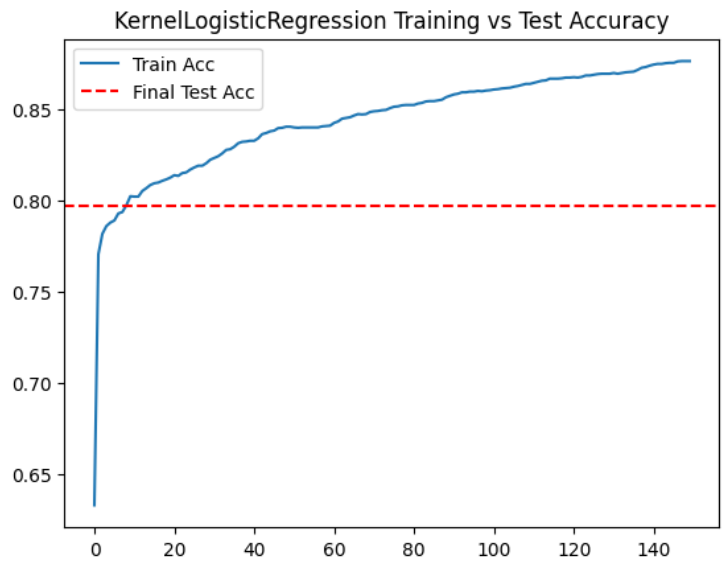

SVM Loss
0.66
0.64
0.62
0.60
0
50
100
150
200
250
300



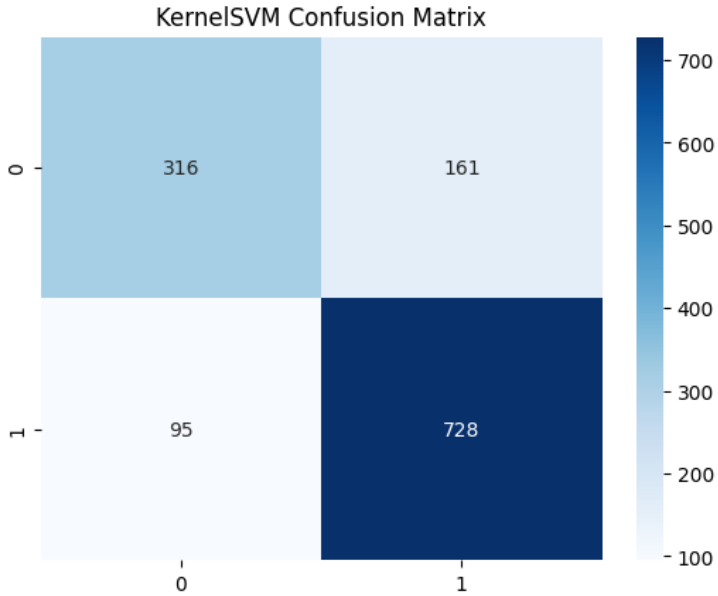

```
False Positives: 222, False Negatives: 125
Type I error (FP rate): 0.171
Type II error (FN rate): 0.096
Model seems reasonably fit.

------------------------------------

KernelLogisticRegression: train_acc=0.877, test_acc=0.797, prec=0.822, rec=0.866, f1=0.844

Classification report:
              precision    recall  f1-score   support

           0       0.75      0.68      0.71       477
           1       0.82      0.87      0.84       823

    accuracy                           0.80      1300
   macro avg       0.78      0.77      0.78      1300
weighted avg       0.79      0.80      0.79      1300
```

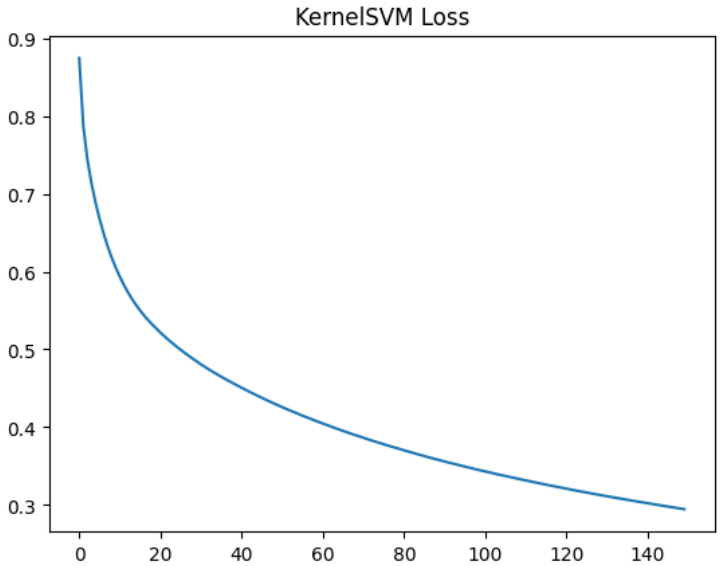

KernelLogisticRegression Confusion Matrix
323
154
110
713
0
1
0
1
700
600
500
400
300
200

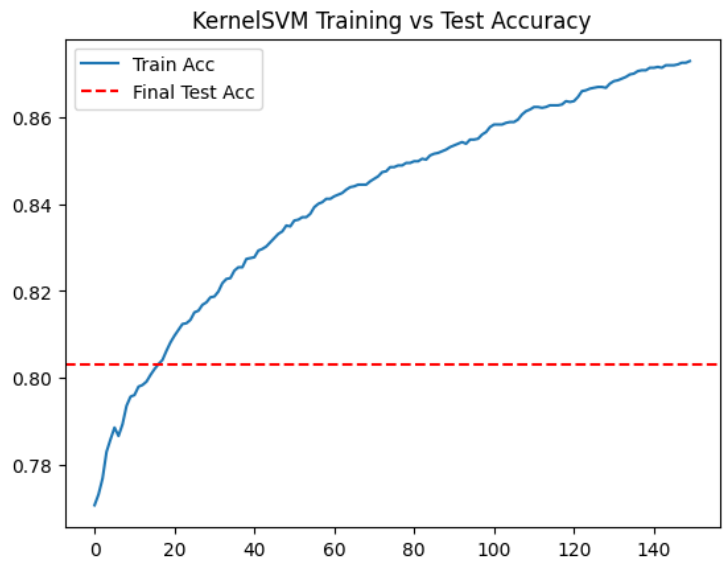

KernelLogisticRegression Loss
0.70
0.65
0.60
0.55
0.50
0.45
0.40
0.35
0
20
40
60
80
100
120
140




```
False Positives: 154, False Negatives: 110
Type I error (FP rate): 0.118
Type II error (FN rate): 0.085
Model seems reasonably fit.

------------------------------------

KernelSVM: train_acc=0.873, test_acc=0.803, prec=0.819, rec=0.885, f1=0.850

Classification report:
               precision    recall  f1-score   support

           0       0.77      0.66      0.71       477
           1       0.82      0.88      0.85       823

    accuracy                           0.80      1300
   macro avg       0.79      0.77      0.78      1300
weighted avg       0.80      0.80      0.80      1300
```


KernelSVM Confusion Matrix
316
161
95
728
0
1
0
1
700
600
500
400
300
200
100


KernelSVM Loss
0.9
0.8
0.7
0.6
0.5
0.4
0.3
0
20
40
60
80
100
120
140




```
False Positives: 161, False Negatives: 95
Type I error (FP rate): 0.124
Type II error (FN rate): 0.073
Model seems reasonably fit.
```

7 Conclusions

From the experimental results, it is evident that kernelized models (Kernel Logistic Regression and Kernel SVM) outperform their linear counterparts (Logistic Regression and linear SVM).
• Linear models (LR and SVM): Achieve test accuracy around 72–73
•Kernelized models (KLR and KSVM): Capture nonlinear relationships in the data, leading to a clear improvement in all metrics. Test accuracy increases to about 80
•Kernel Logistic Regression reduces false positives and achieves a strong balance between precision (0.822) and recall (0.866).
•Kernel SVM achieves the highest recall (0.885) and thus the lowest false negative rate, making it the best choice when minimizing Type II errors is critical.
•Classification errors: Kernelized models reduce both Type I errors (FP rate 0.12 vs 0.17 for linear models) and Type II errors (FN rate as low as 0.073 for Kernel SVM).

While linear models provide a solid baseline, kernelized models deliver a significant performance boost. Among all, the Kernel SVM stands out as the best-performing model, effectively balancing precision and recall while minimizing critical errors.

7.1 Performances

From a computational performance perspective, the individual code blocks for training both linear and kernelized models are extremely fast, with execution times generally under 0.1 seconds. This demonstrates that the from-scratch implementations are efficient and suitable for moderately sized datasets.

The grid search, however, is more time-consuming, taking approximately 14 minutes, as it systematically evaluates all hyperparameter combinations using 3-fold cross-validation. Similarly, the evaluation phase of the four models on the test set took about 2 and a half minutes, mainly due to metric computation, confusion matrix generation, and plotting.

In summary, the project highlights that the from-scratch models are very fast for training, while hyperparameter tuning and comprehensive evaluation require more time due to the computational complexity of systematic parameter search and detailed output generation.

I declare that this material, which I now submit for assessment, is entirely my own work and has not been taken from the work of others, save and to the extent that such work has been cited and acknowledged within the text of my work. I understand that plagiarism, collusion, and copying are grave and serious offences in the university and accept the penalties that would be imposed should I engage in plagiarism, collusion or copying. This assignment, or any part of it, has not been previously submitted by me or any other person for assessment on this or any other course of study.